

Week 9 (Thursday, August 21)

Neural Networks and Deep Learning

CMPE-257: Machine Learning - Summer 2025



SJSU SAN JOSÉ STATE
UNIVERSITY

Instructor: Rafael E Banchs
Email: rafael.banchs@sjsu.edu

Today's plan...

Main Topic: Neural Networks and Deep Learning

- **5:30-6:00** Session 1
- **6:00-7:00** Quiz 3
- **7:00-7:40** Session 1 (continuation)
- **7:40-8:00** Break
- **8:00-9:20** Session 2
- **9:20-9:30** Term Projects and HW9

Session 1:

- Neural Networks

Session 2:

- Deep Learning
- Hands-on coding session 9

**Eight Homework
is due on Sunday!**

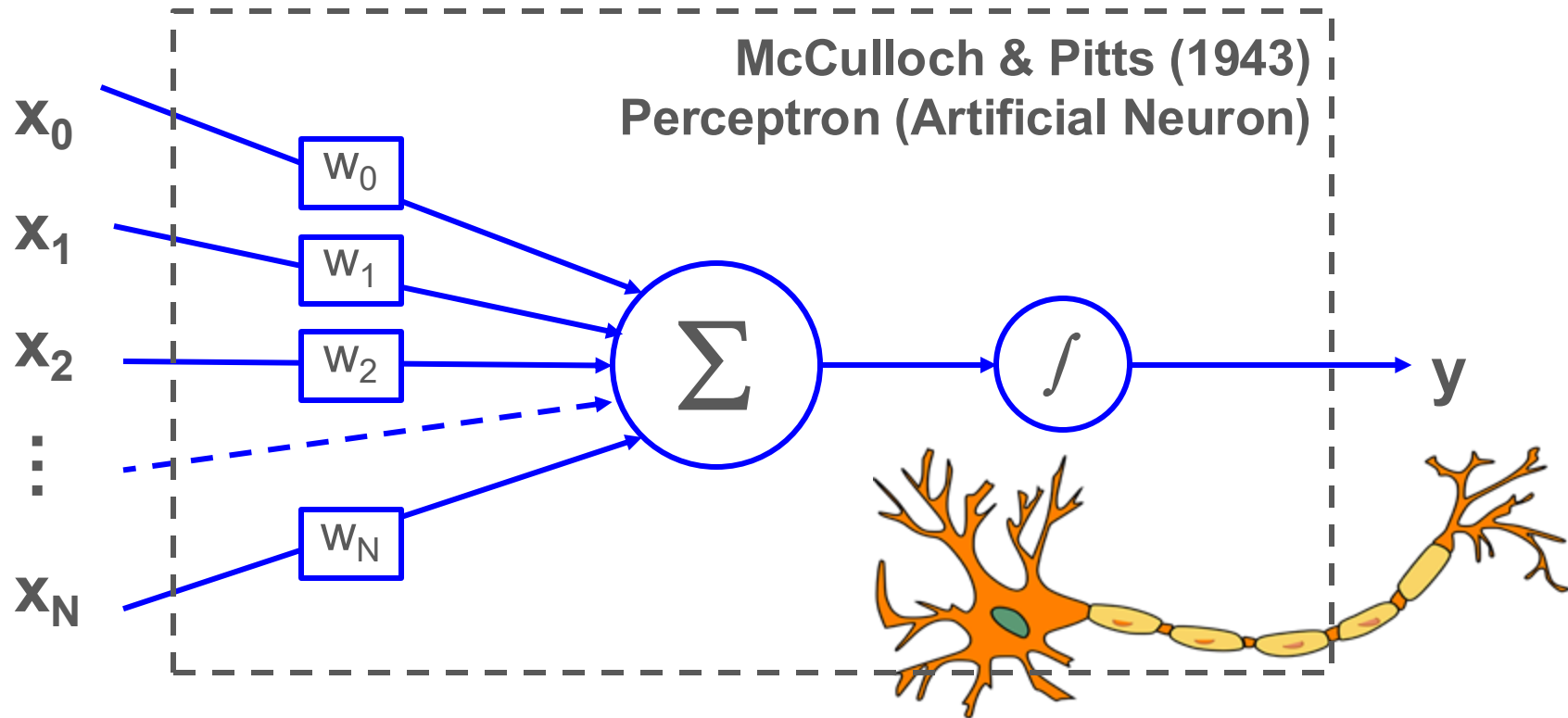
Week 9 - Session 1

Neural Networks

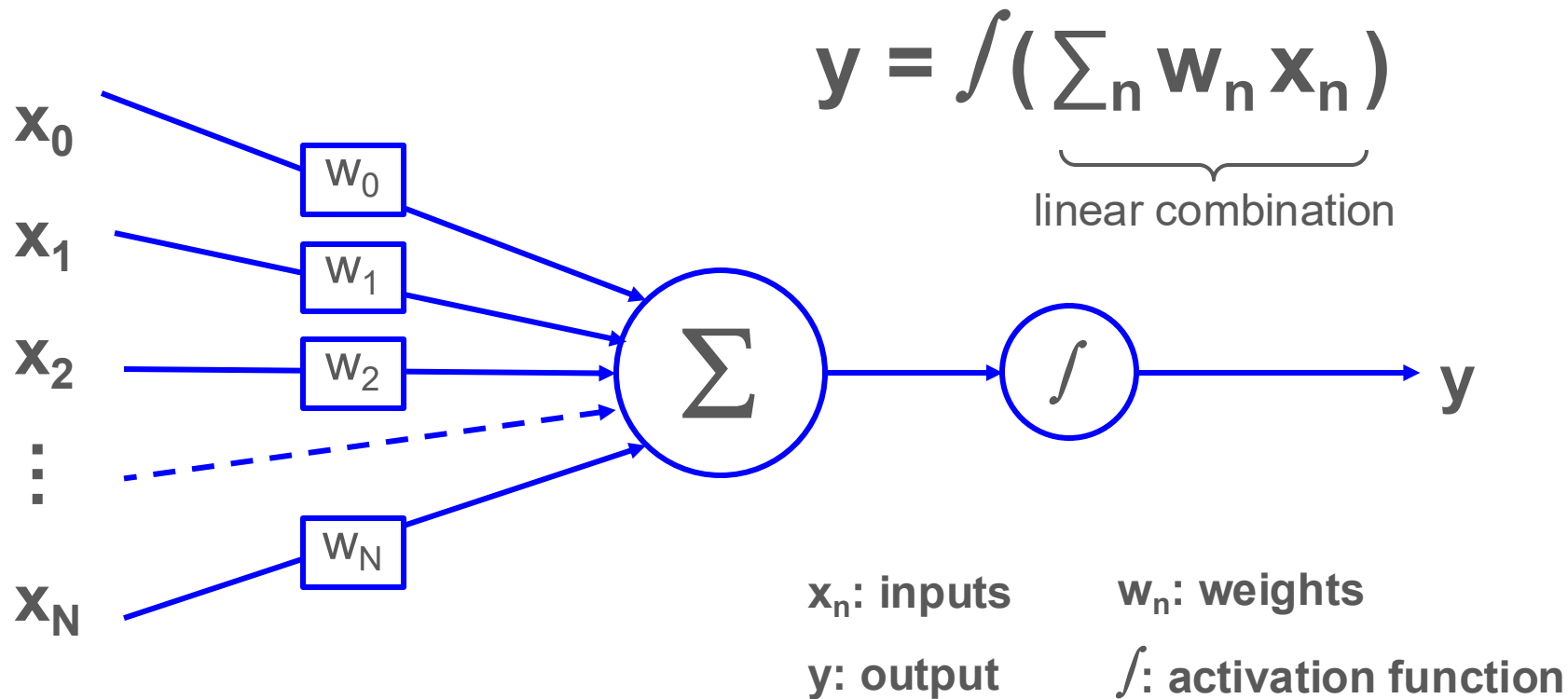
- Multilayer Perceptron
- Backpropagation
- Gradient Descent Algorithms



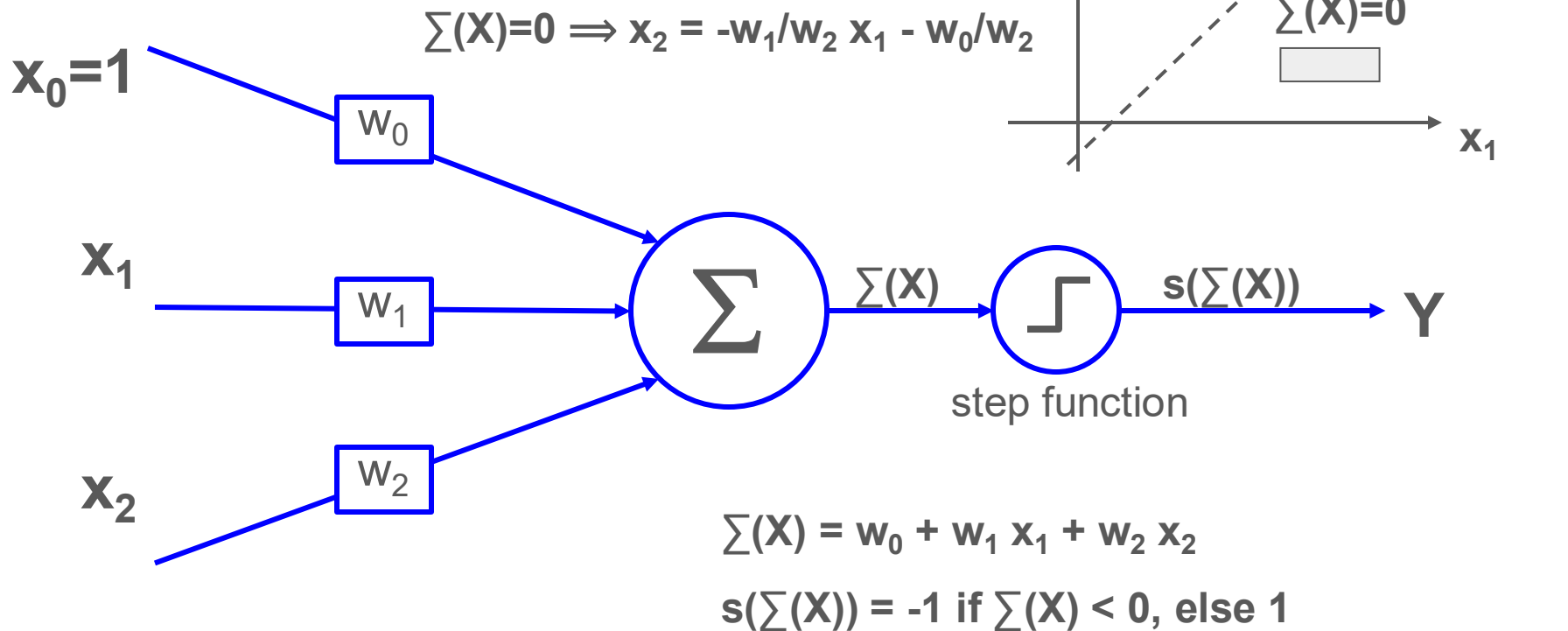
The perceptron: a model for an artificial neuron



The perceptron: a linear combinator with an activation



The perceptron as a binary classifier





Boolean logic with perceptrons

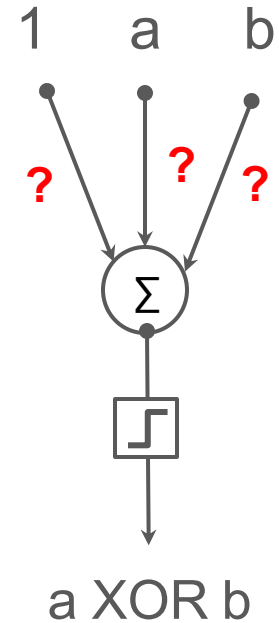
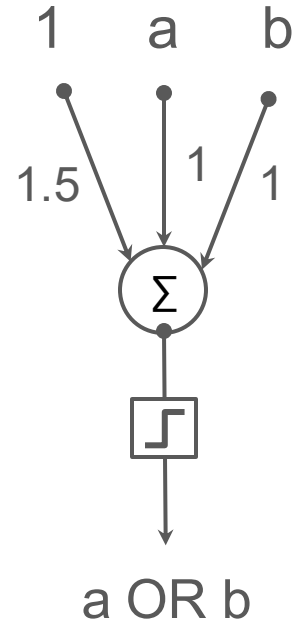
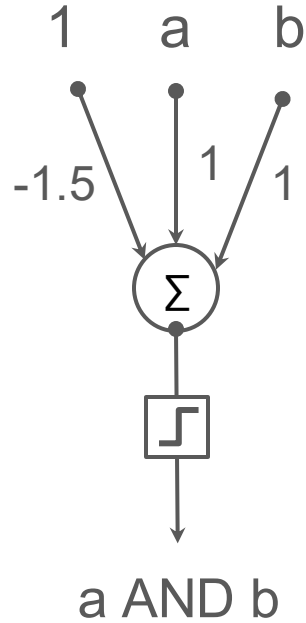
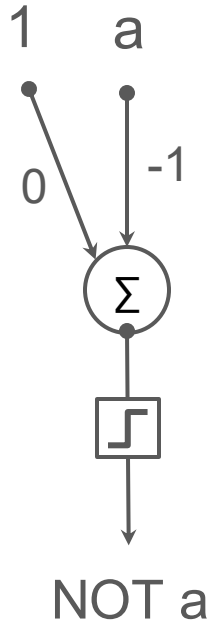
- Consider the boolean operations described below:

a	b	NOT a	a AND b	a OR b	a XOR b
-1	-1	+1	-1	-1	-1
-1	+1	+1	-1	+1	+1
+1	-1	-1	-1	+1	+1
+1	+1	-1	+1	+1	-1

- Use perceptrons to implement each of them (one perceptron per operation)

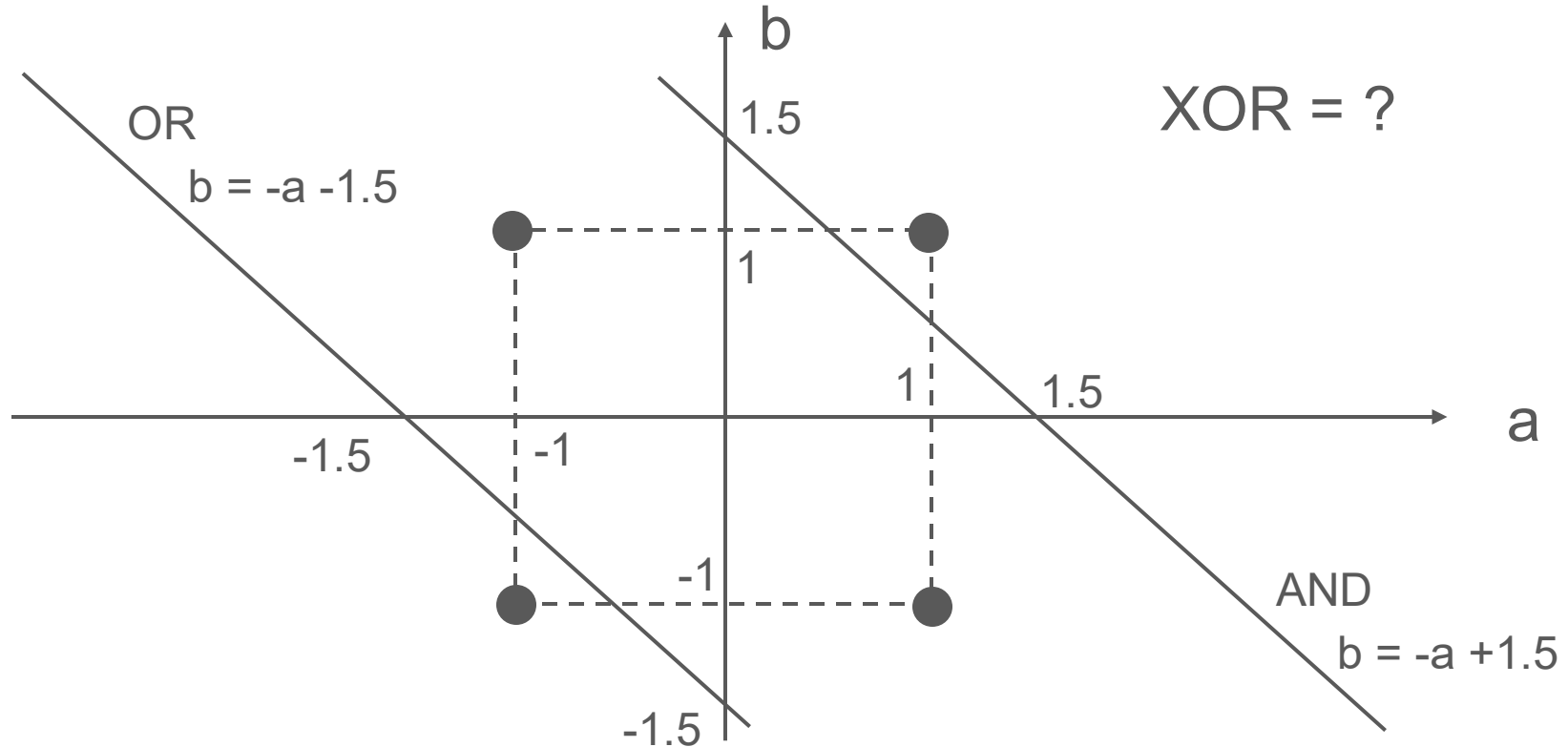


Boolean logic with perceptrons



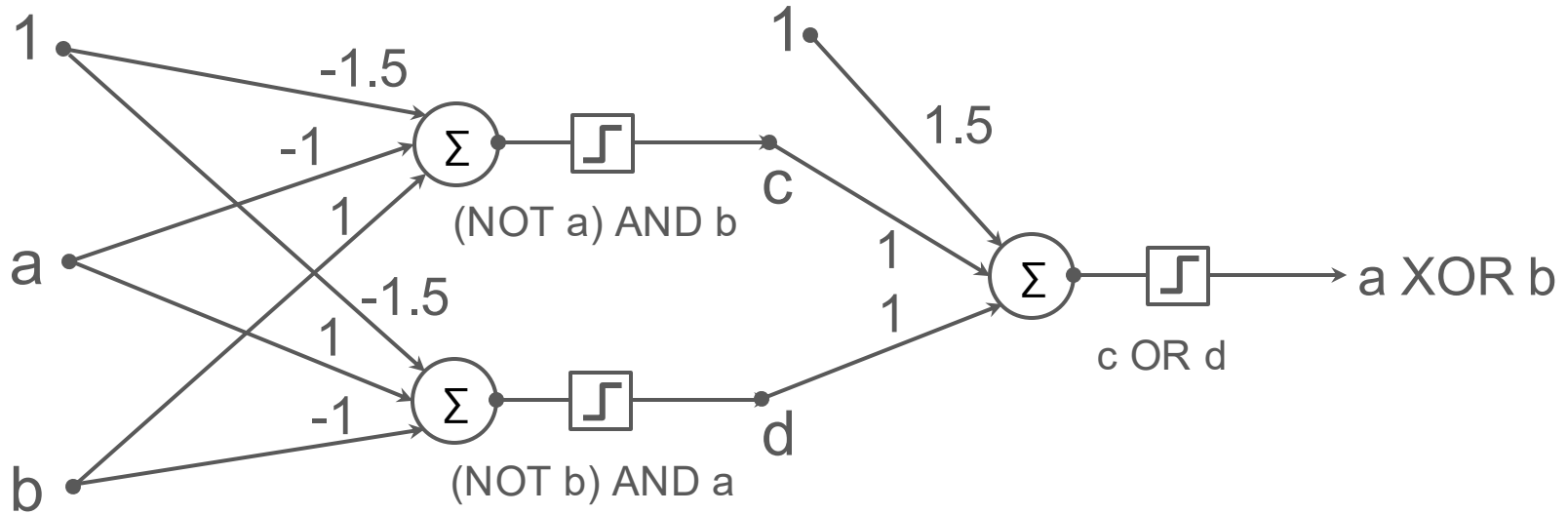


Boolean logic with perceptrons



Implementing XOR with perceptrons

- $a \text{ XOR } b = ((\text{NOT } a) \text{ AND } b) \text{ OR } (a \text{ AND } (\text{NOT } b))$



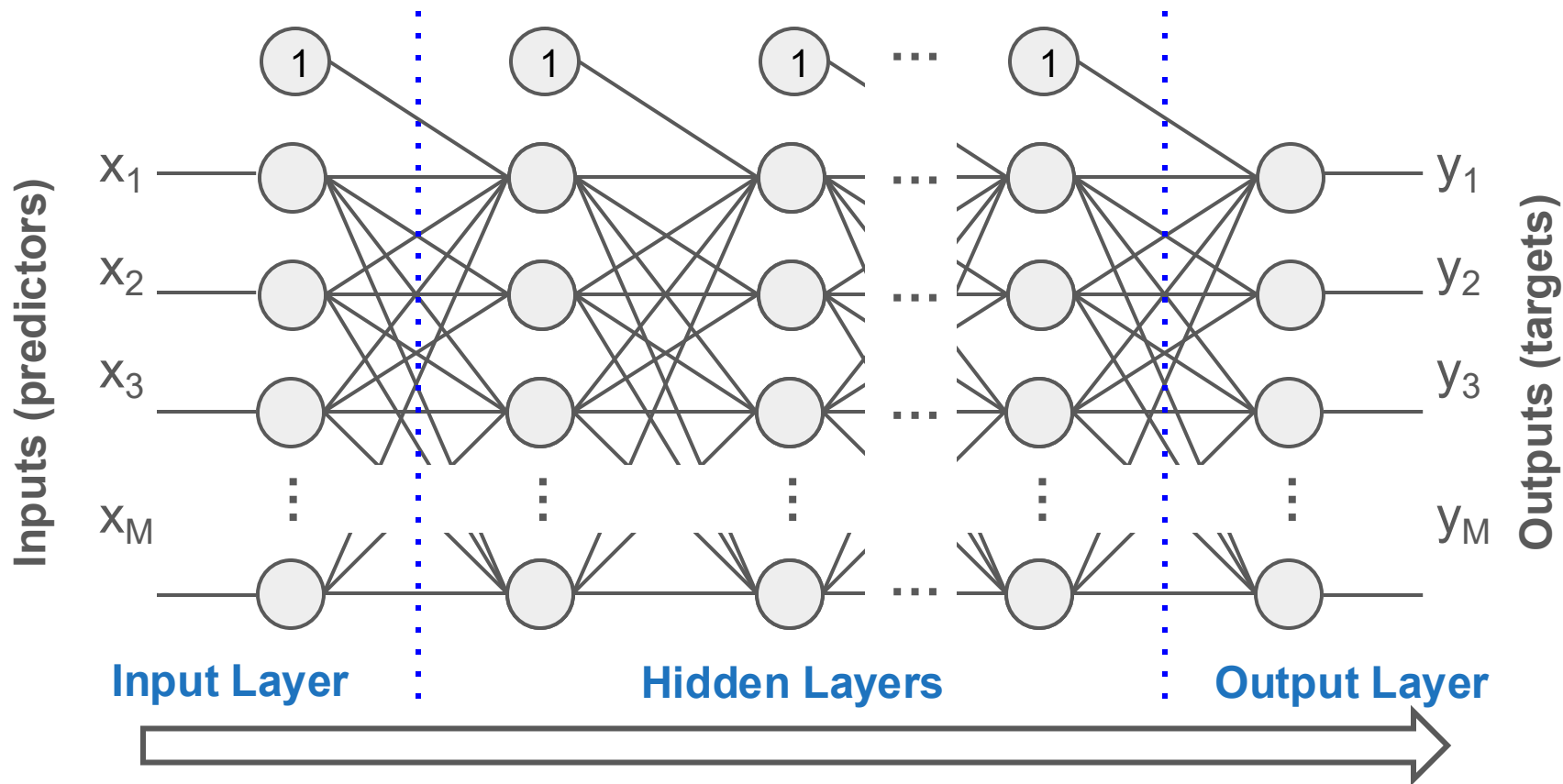
QUIZ-TIME

Third Quiz



60 minutes

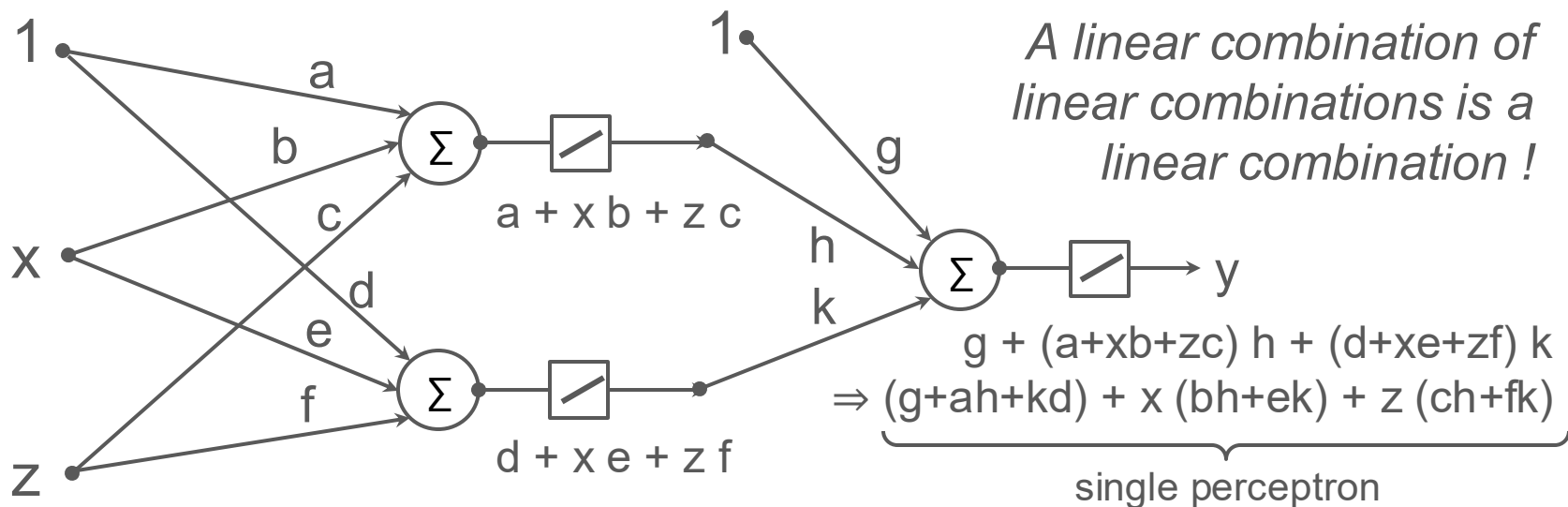
The multilayer perceptron (MLP)





The linear MLP problem

- Show that any MLP with only linear activation functions (identity function) can be reduced to a single perceptron





Generalized regression with augmented predictors

- In general, any smooth and continuous arbitrary decision boundary can be implemented with a three-layer MLP
- Given input nodes $1, x_1 \dots x_M$ and a hidden layer with K nonlinear units with outputs $h_k(\mathbf{w}_k^T \mathbf{x})$, a node j in the output layer with linear activation will produce:

$$y_j = \underbrace{w_{0j}}_{\text{Bias Term}} + \sum_k w_{kj} \underbrace{h_k(\mathbf{w}_k^T \mathbf{x})}_{\text{Augmented predictor space}}$$

Learned from data !



Learning MLP weights from data

- For a perceptron with hard-threshold activation function, learning the weights is a hard combinatorial problem
- For an MLP with hard-thresholds the problem is harder!
- As seen before, soft-threshold activation functions make the problem of estimating weights more tractable
- For a single perceptron the error surface is convex
- For an MLP the the error surface is not convex anymore!

Soft-threshold activation functions

- In general, MLP units might implement multiple different types of activation functions
- Traditionally, hyperbolic tangent was commonly used as the default activation function in MLPs

$$\tanh(x) = \frac{\exp(x) - \exp(-x)}{\exp(x) + \exp(-x)} = \frac{\exp(2x) - 1}{\exp(2x) + 1}$$

- It exhibits a sigmoid behavior and other nice properties

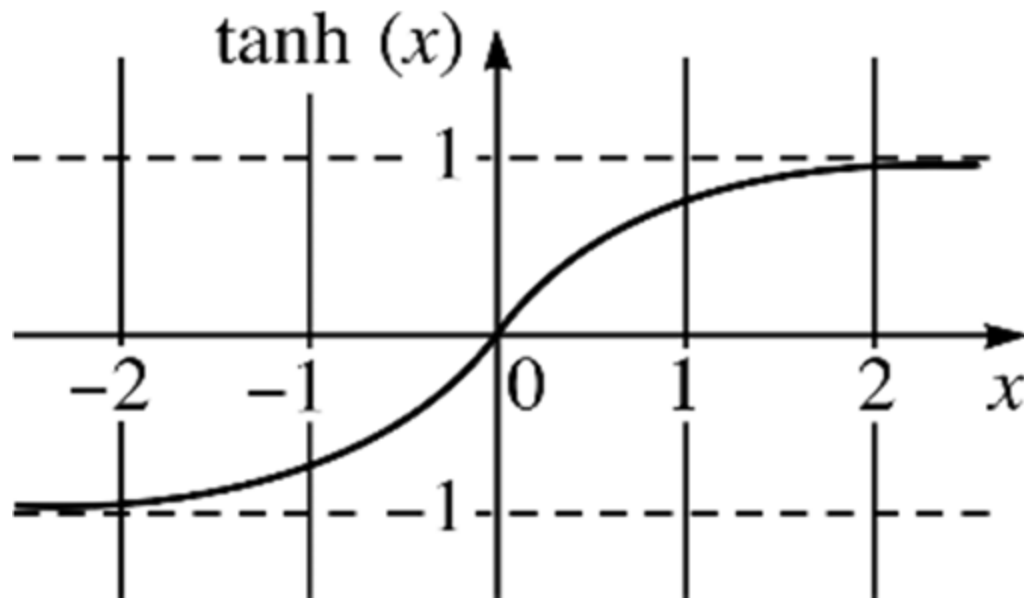


Hyperbolic tangent function

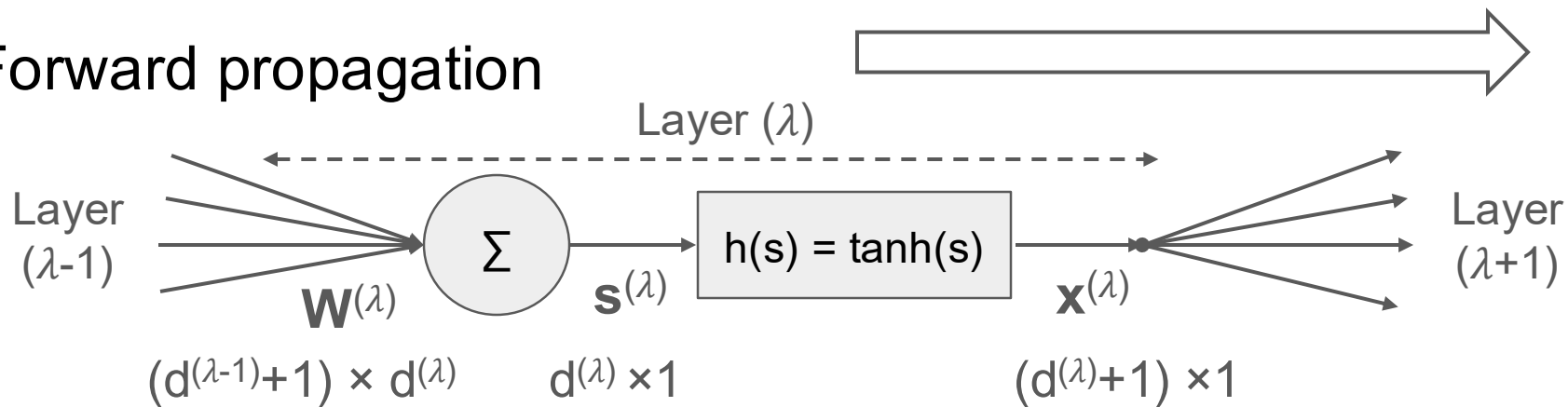
$$\tanh(x) = \frac{\exp(2x) - 1}{\exp(2x) + 1}$$

- $\tanh(0) = 0$
- $\tanh(x)|_{x \gg 0} \rightarrow 1$
- $\tanh(x)|_{x \ll 0} \rightarrow -1$

$$\frac{d}{dx} \tanh(x) = 1 - \tanh^2(x)$$



Forward propagation



$d^{(\lambda)}$: number of units in layer (λ)

- Initialization: $\mathbf{x}^{(0)} \leftarrow \mathbf{x}$
- Forward propagation loop:
for $= 1, 2, 3, \dots L$: $\mathbf{s}^{(\lambda)} \leftarrow \mathbf{W}^{(\lambda)T} \mathbf{x}^{(\lambda-1)}$; $\mathbf{x}^{(\lambda)} \leftarrow \begin{pmatrix} 1 \\ h(\mathbf{s}^{(\lambda)}) \end{pmatrix}$
- Output: $\hat{\mathbf{y}} \leftarrow \mathbf{x}^{(L)}$

Error and gradient computation

- Overall error: $E_{in}(\mathbf{w}) = 1/N \sum_n e_n$

**Sensitivity Vector
for layer λ**

- Partial derivatives: $\frac{\partial E_{in}}{\partial \mathbf{W}^{(\lambda)}} = \frac{1}{N} \sum_n \frac{\partial e_n}{\partial \mathbf{W}^{(\lambda)}}$

- By applying the chain rule: $\frac{\partial e}{\partial \mathbf{W}^{(\lambda)}} = \mathbf{x}^{(\lambda-1)} (\boldsymbol{\delta}^{(\lambda)})^T$

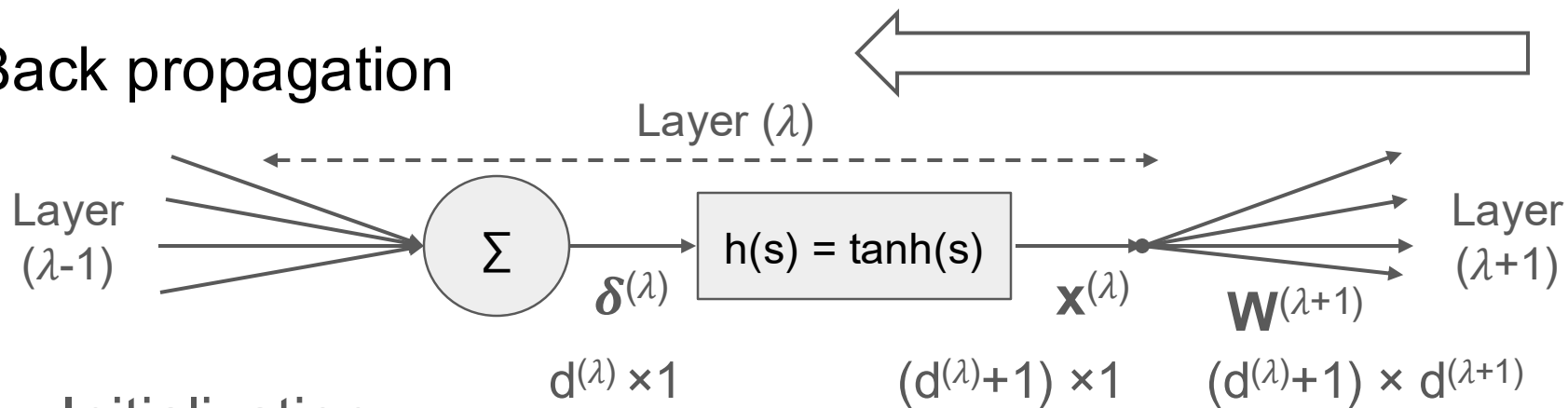
Matrix of $(d^{(\lambda-1)}+1) \times d^{(\lambda)}$

- Where $\boldsymbol{\delta}^{(\lambda)} = \frac{\partial e}{\partial \mathbf{s}^{(\lambda)}}$ and $\mathbf{x}^{(\lambda-1)} = \frac{\partial \mathbf{s}^{(\lambda)}}{\partial \mathbf{W}^{(\lambda)}}$ Remember...
 $\mathbf{s}^{(\lambda)} = \mathbf{W}^{(\lambda)T} \mathbf{x}^{(\lambda-1)}$

Propagating back the sensitivities

- In the forward pass $\mathbf{x}^{(\lambda)}$ was computed from $\mathbf{x}^{(\lambda-1)}$
- A backward pass can be used to compute $\delta^{(\lambda)}$ from $\delta^{(\lambda+1)}$ starting from $\delta^{(L)} = \partial e / \partial \mathbf{s}^{(L)} = 2 (\mathbf{x}^{(L)} - \mathbf{y}) \partial \mathbf{x}^{(L)} / \partial \mathbf{s}^{(L)}$
- In the hidden layers: $\delta_j^{(\lambda)} = \partial e / \partial \mathbf{s}_j^{(\lambda)} = \partial e / \partial \mathbf{x}_j^{(\lambda)} \partial \mathbf{x}_j^{(\lambda)} / \partial \mathbf{s}_j^{(\lambda)}$
 - $\partial e / \partial \mathbf{x}_j^{(\lambda)} = \sum_k \partial e / \partial \mathbf{s}_k^{(\lambda+1)} \partial \mathbf{s}_k^{(\lambda+1)} / \partial \mathbf{x}_j^{(\lambda)} = \sum_k w_{jk}^{(\lambda+1)} \delta_j^{(\lambda+1)}$
 - $\partial \mathbf{x}_j^{(\lambda)} / \partial \mathbf{s}_j^{(\lambda)} = 1 - \tanh^2(\mathbf{s}_j^{(\lambda)}) = 1 - (\mathbf{x}_j^{(\lambda)})^2$
- $\delta^{(\lambda)} = [\mathbf{1} - \mathbf{x}^{(\lambda)} \otimes \mathbf{x}^{(\lambda)}] \otimes [\mathbf{W}^{(\lambda+1)} \delta^{(\lambda+1)}]^*$ (*excluding the bias term)

Back propagation



- Initialization:

- If $h_L(s) = s : \delta^{(L)} \leftarrow 2 (\mathbf{x}^{(L)} - \mathbf{y})$
- If $h_L(s) = \tanh(s) : \delta^{(L)} \leftarrow 2 (\mathbf{x}^{(L)} - \mathbf{y}) \otimes (1 - (\mathbf{x}^{(L)})^2)$

- Back propagation loop:

for $\lambda = L-1, L-2, \dots, 1$: $\delta^{(\lambda)} = [1 - \mathbf{x}^{(\lambda)} \otimes \mathbf{x}^{(\lambda)}] \otimes [\mathbf{W}^{(\lambda+1)} \delta^{(\lambda+1)}]^*$

(*excluding the bias term)



Updating the weights with backpropagation

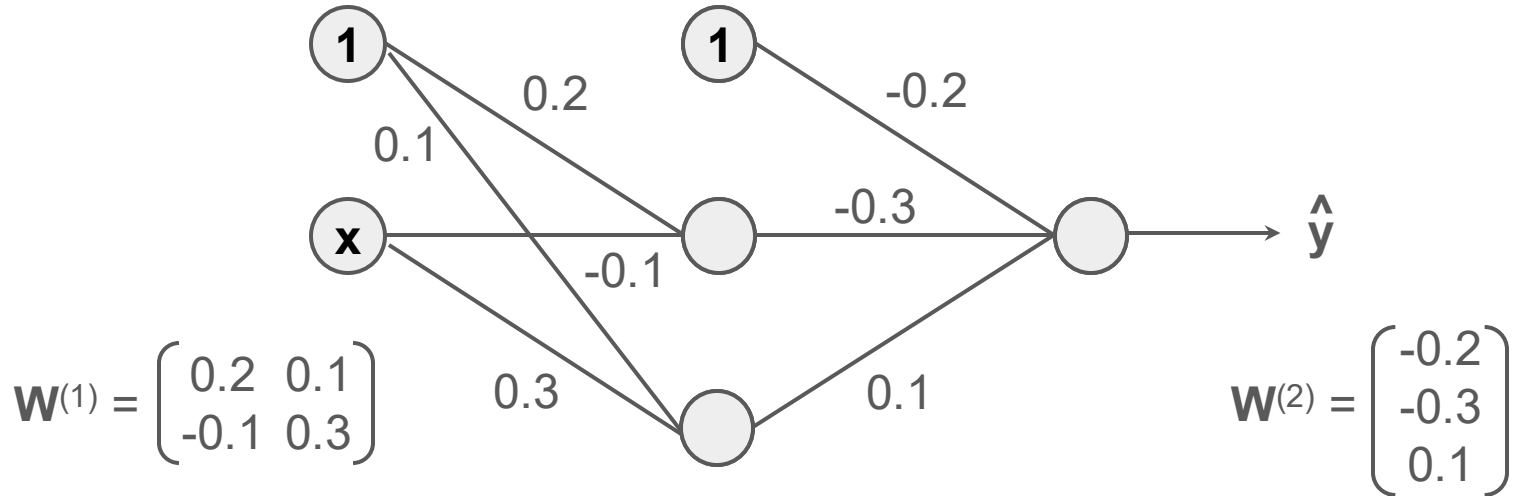
- Initialization: $\mathbf{G}^{(\lambda)} = \mathbf{0}$ $\mathbf{W}^{(\lambda)}$ for $\lambda = 1, 2, 3, \dots L$
- For each training data sample $(\mathbf{x}_n, \mathbf{y}_n)$ with $n = 1, 2, \dots N$:
 - compute $\mathbf{x}^{(\lambda)}$ for $\lambda = 0, 1, \dots L$ (forward propagation)
 - compute $\delta^{(\lambda)}$ for $\lambda = L, L-1, \dots 1$ (backpropagation)
 - For $\lambda = 1, \dots L$: $\mathbf{G}^{(\lambda)} \leftarrow \mathbf{G}^{(\lambda)} + 1/N (\mathbf{x}^{(\lambda-1)} (\delta^{(\lambda)})^T)$
- For $\lambda = 1, 2, 3, \dots L$: $\mathbf{W}^{(\lambda)} \leftarrow \mathbf{W}^{(\lambda)} - \eta \mathbf{G}^{(\lambda)}$

Gradient descent!



Back propagation exercise

- Consider the neural network below with activations $\tanh()$, compute $\partial e / \partial \mathbf{W}^{(\lambda)}$ for $\lambda=1,2$ and the data point $(x=3, y=1)$





Back propagation exercise

- Forward propagation

$$\mathbf{W}^{(1)} = \begin{pmatrix} 0.2 & 0.1 \\ -0.1 & 0.3 \end{pmatrix} \quad \mathbf{W}^{(2)} = \begin{pmatrix} -0.2 \\ -0.3 \\ 0.1 \end{pmatrix}$$

$\mathbf{x}^{(0)}$	$\mathbf{s}^{(1)}$	$\mathbf{x}^{(1)}$	$\mathbf{s}^{(2)}$	$\mathbf{x}^{(2)}$
$[1 \ 3]^T$	$\mathbf{W}^{(1)T} \mathbf{x}^{(0)}$	$[1 \ \tanh(\mathbf{s}^{(1)T})^T]^T$	$\mathbf{W}^{(2)T} \mathbf{x}^{(1)}$	$\tanh(\mathbf{s}^{(2)})$
$\begin{pmatrix} 1 \\ 3 \end{pmatrix}$	$\begin{pmatrix} -0.1 \\ 1.0 \end{pmatrix}$	$\begin{pmatrix} 1.00 \\ -0.10 \\ 0.76 \end{pmatrix}$	$[-0.094]$	$[-0.094]$



Back propagation exercise

- Backpropagation

$$\mathbf{W}^{(1)} = \begin{pmatrix} 0.2 & 0.1 \\ -0.1 & 0.3 \end{pmatrix} \quad \mathbf{W}^{(2)} = \begin{pmatrix} -0.2 \\ -0.3 \\ 0.1 \end{pmatrix}$$

$\delta^{(2)}$	$\delta^{(1)}$
$2 (\mathbf{x}^{(2)} - \mathbf{y}) (1 - (\mathbf{x}^{(2)})^2)$	$[1 - \mathbf{x}^{(1)} \otimes \mathbf{x}^{(1)}] \otimes [\mathbf{W}^{(2)} \delta^{(2)}]$
$= 2 (-0.094 - 1) (1 - (-0.094)^2)$ $= -2.1687$	$= \begin{pmatrix} 0.99 \\ 0.42 \end{pmatrix} \otimes \begin{pmatrix} 0.65 \\ -0.22 \end{pmatrix} = \begin{pmatrix} 0.64 \\ -0.09 \end{pmatrix}$



Back propagation exercise

- Computation of partial derivatives

$\partial e / \partial \mathbf{W}^{(1)} = \mathbf{x}^{(0)} (\boldsymbol{\delta}^{(1)})^T$	$\partial e / \partial \mathbf{W}^{(2)} = \mathbf{x}^{(1)} (\boldsymbol{\delta}^{(2)})^T$
$\begin{pmatrix} 1 \\ 3 \end{pmatrix} [0.64 \ -0.09] = \begin{pmatrix} 0.64 & 1.92 \\ -0.09 & -0.27 \end{pmatrix}$	$\begin{pmatrix} 1.00 \\ -0.10 \\ 0.76 \end{pmatrix} [-2.1687] = \begin{pmatrix} -2.1687 \\ 0.2169 \\ -1.6482 \end{pmatrix}$

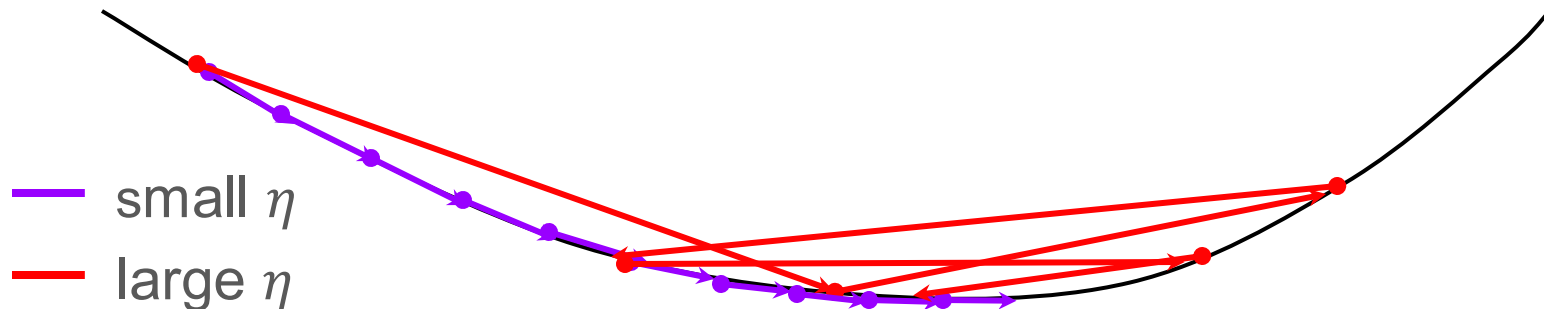


Gradient descent revisited

- Start with an initial set of weights $\mathbf{w}(t)$ at time step $t=0$
 - randomly generated (normal distribution with zero mean and small variance)
- Compute the gradient $\nabla E(\mathbf{w}(t))$ and update the weights moving in the opposite gradient direction: $\mathbf{w}(t+1) = \mathbf{w}(t) - \eta \nabla E(\mathbf{w}(t))$
- Keep iterating until termination criteria is satisfied
 - A maximum number of iterations
 - Marginal error improvement (i.e. small gradient)
 - Small error value over a few iterations

Limitation of the vanilla gradient descent algorithm

- Standard gradient descent is limited due to the use of a fixed learning rate η :
 - If too small a lot of steps will be needed to converge
 - If too large it can overshoot the solution and oscillate



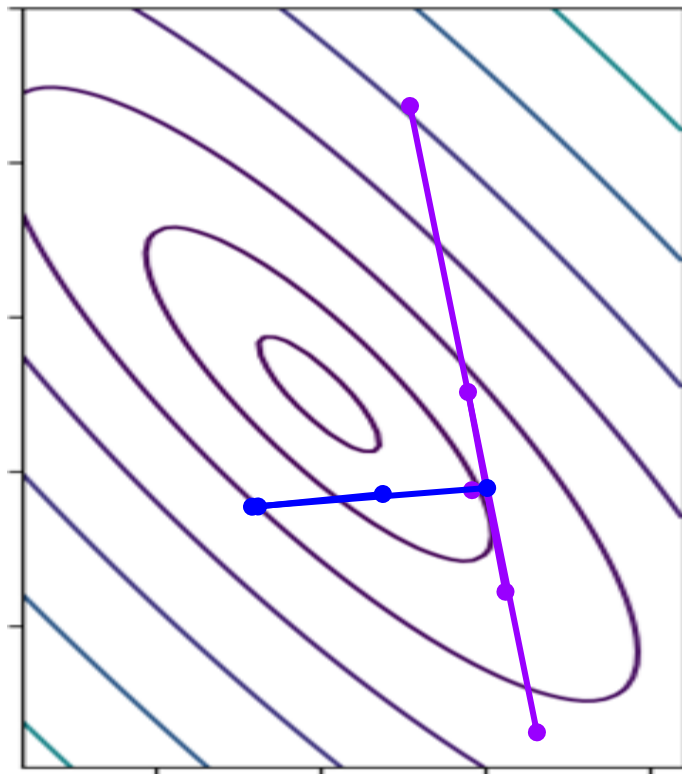
Gradient descent enhancements

Steepest Descent:

- Gradient descent with line search (i.e. find the best η at each step)

$$\operatorname{argmin}_{\eta} E_{\text{in}}(\mathbf{w}(t) - \eta \nabla E(\mathbf{w}(t)))$$

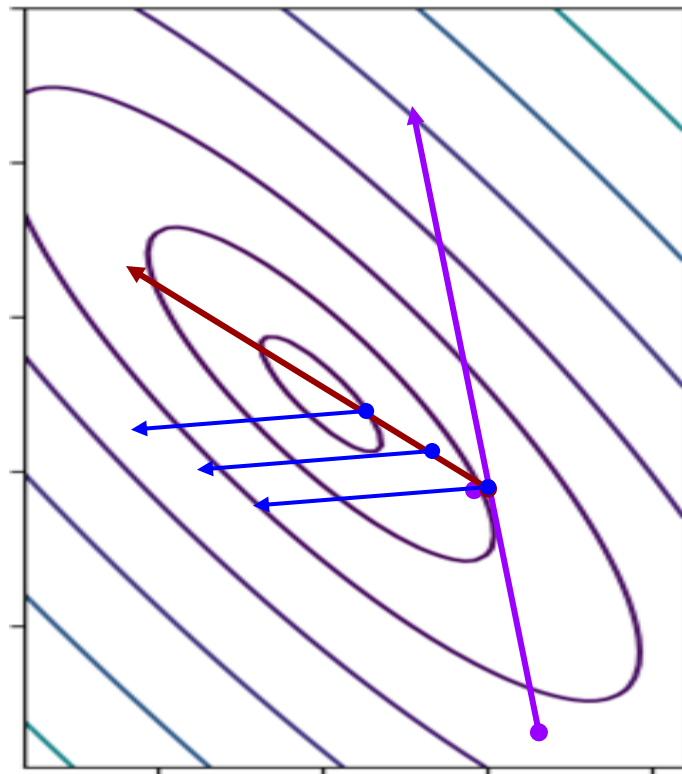
- Implements an efficient line search by means of a bisection algorithm



Gradient descent enhancements

Conjugate Gradient Descent:

- Move into the conjugate direction at each new step
- I.e. the direction along which the gradient is perpendicular to the previous search directions
- Improves convergence significantly!



Neural networks and overfitting

- In general, neural network architectures involve a large number of parameters (high complexity)
- In general, gradient descent algorithms allow for a good fitting of the training data
- Accordingly, overfitting is most likely to happen!
- Different mechanisms allow to prevent overfitting when training neural network based models

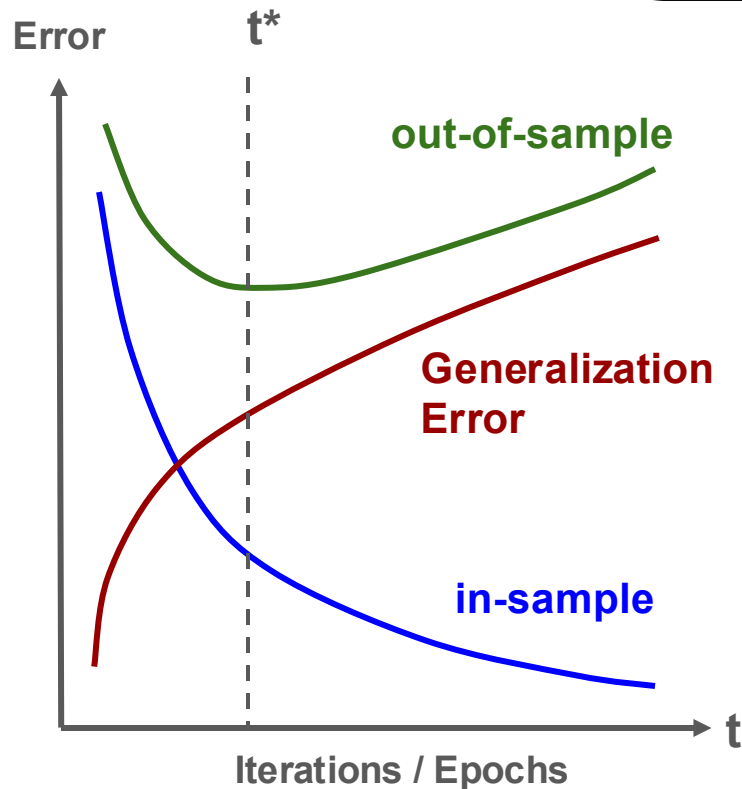
Regularization via weight penalization

- Augmented error functions can be used for regularization:
 - Squared weight decay penalization:
$$E_{\text{aug}}(\mathbf{w}, \alpha) = E_{\text{in}}(\mathbf{w}) + \alpha \sum_{\lambda, i, j} (w_{i, j}^{(\lambda)})^2$$
 - Weight elimination (small weights decay faster):
$$E_{\text{aug}}(\mathbf{w}, \alpha) = E_{\text{in}}(\mathbf{w}) + \alpha \sum_{\lambda, i, j} (w_{i, j}^{(\lambda)})^2 / (1 + (w_{i, j}^{(\lambda)})^2)$$
- Cross-validation is used to select a good value for the regularization parameter α



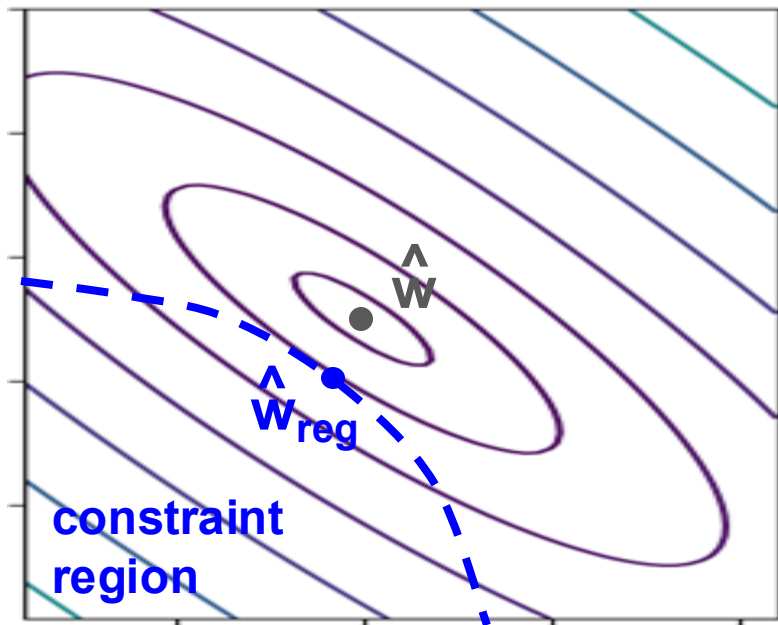
Regularization via early stopping

- Gradient descent fits the training data progressively in an iterative fashion
- A validation set is used to track the out-of-sample error
- Return the model with smallest out-of-sample error

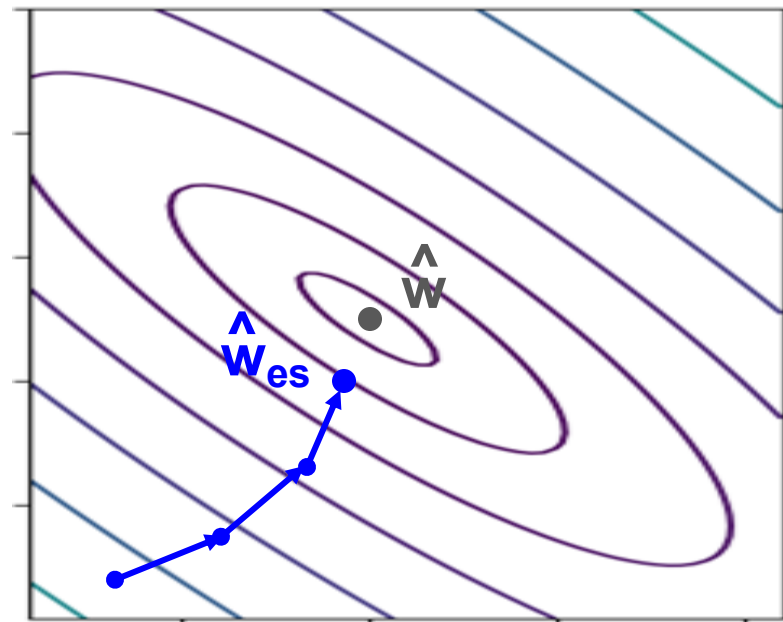


Early stopping vs weight penalization

Penalization

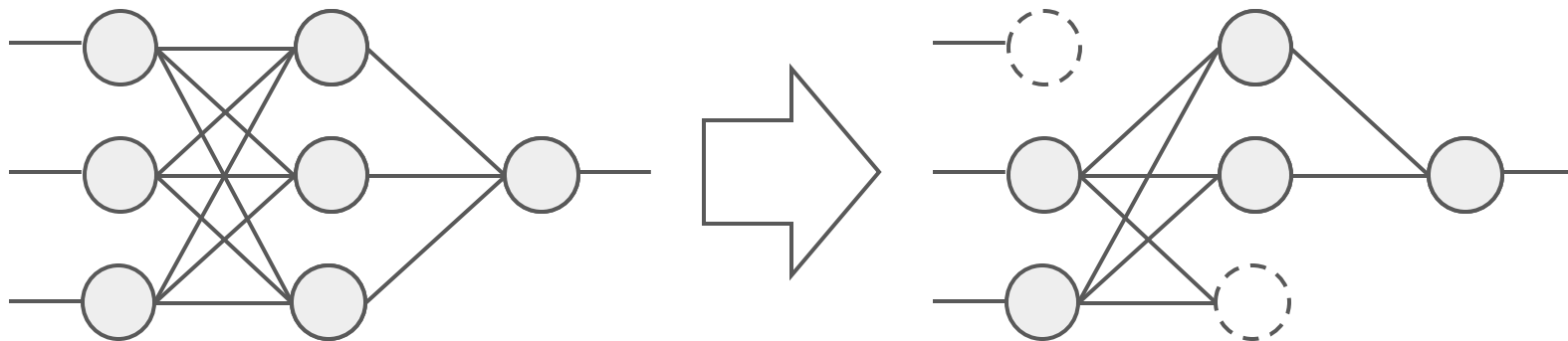


Early Stopping



Regularization via pruning (dropout)

- Randomly removes a small fraction of the units at each iteration when fitting the model
- In practice this is done by setting to zero the activation function of those units selected for dropout



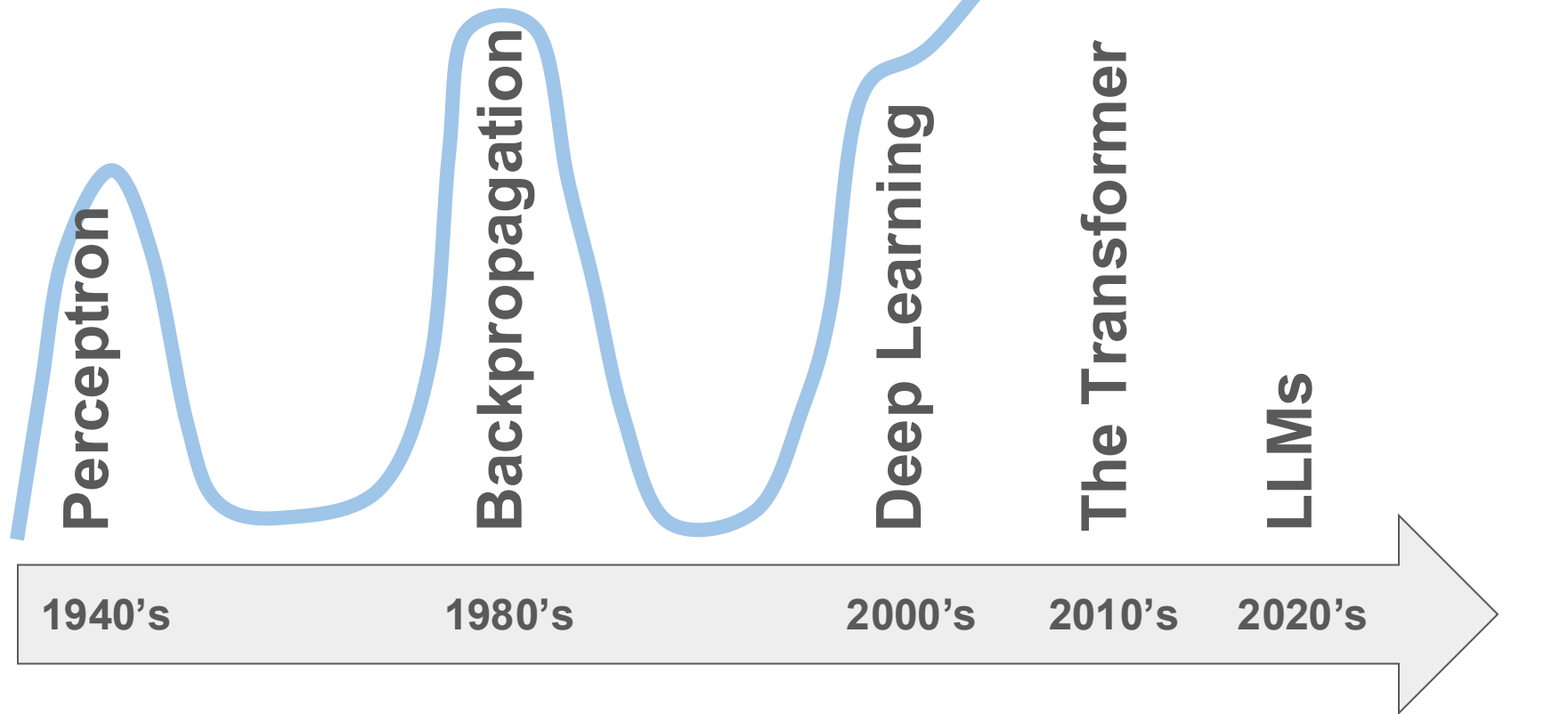


Week 9 - Session 2

Deep Learning

- Convolutional Neural Networks
- Recurrent Neural Networks
- The Transformer

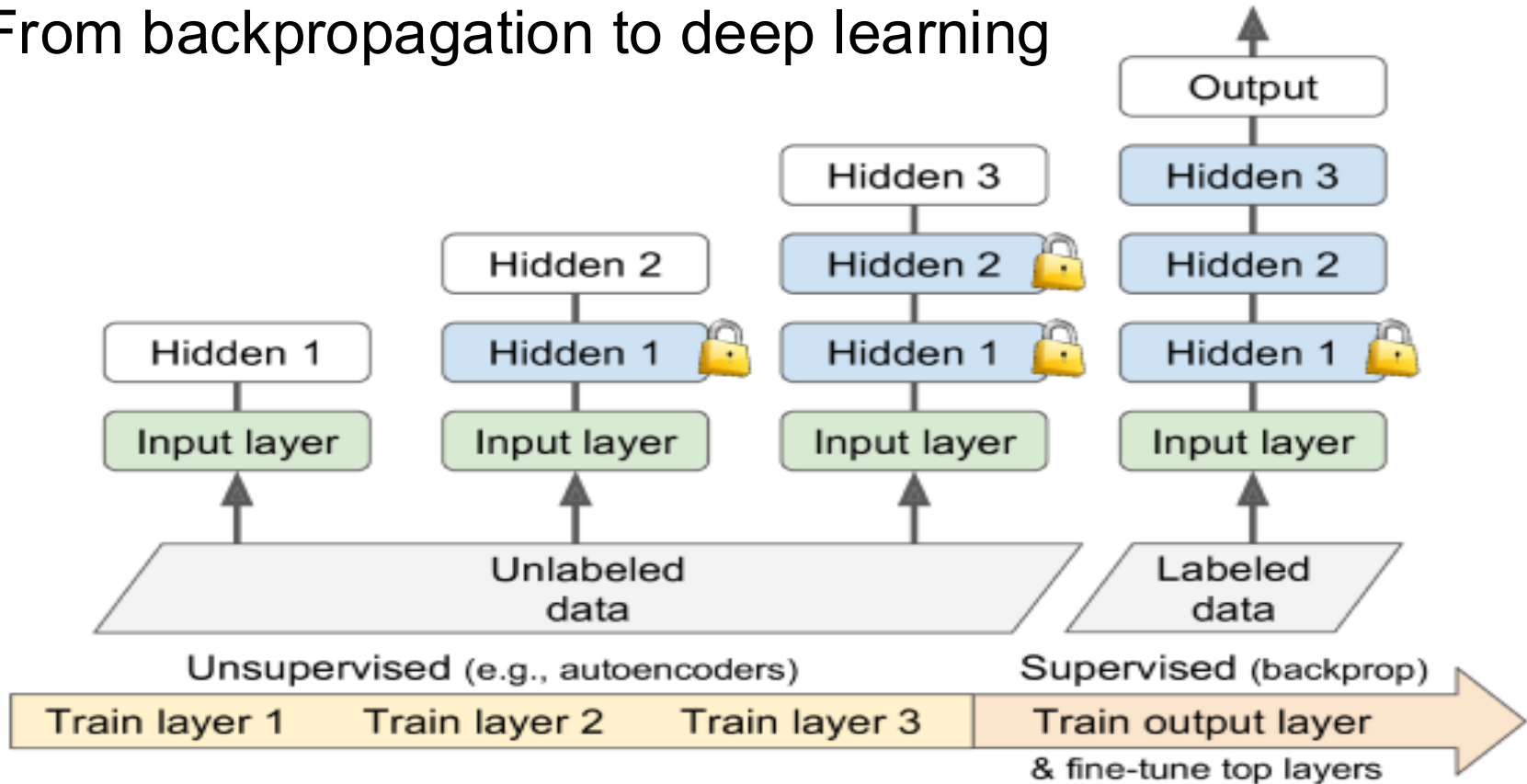
A little bit of history...



From backpropagation to deep learning

- Training MLPs with many hidden layers ($L > 3$) was challenging:
 - Vanishing and exploding gradients
 - Nonconvex optimization
 - Poor initialization / sensitivity
 - Limited computational resources
- An effective method to train “deep” MLPs was proposed:
 - Step 1: pretrain the different layers in an unsupervised fashion (one layer at a time)
 - Step 2: fine-tune the whole network with backpropagation

From backpropagation to deep learning



Accounting for spatial dependencies

- Convolutional Neural Networks (CNNs):
 - Convolution filter layers (pattern detection)
 - (Max) pooling layers (summarization)
 - Fully-connected layers (classification)

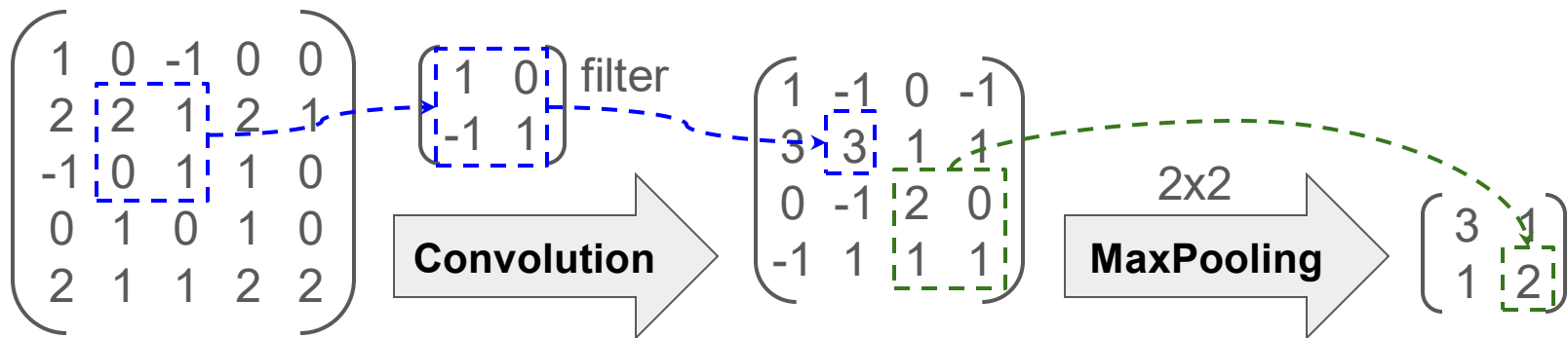
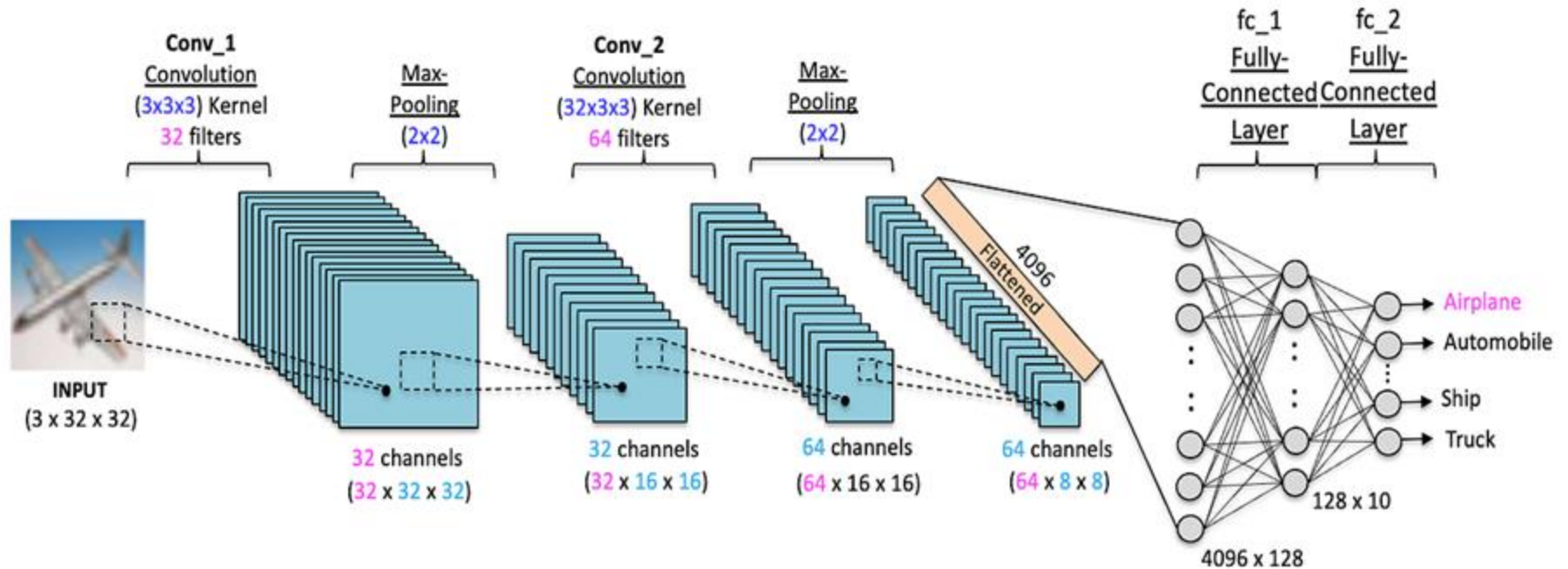


Image classification with CNNs



https://medium.com/@vinith_aswath/demystifying-convolutional-neural-networks-cnns-seeing-the-world-like-a-computer-163c06af4db6



Convolution and pooling exercise...

- Consider the following data matrix and filters:

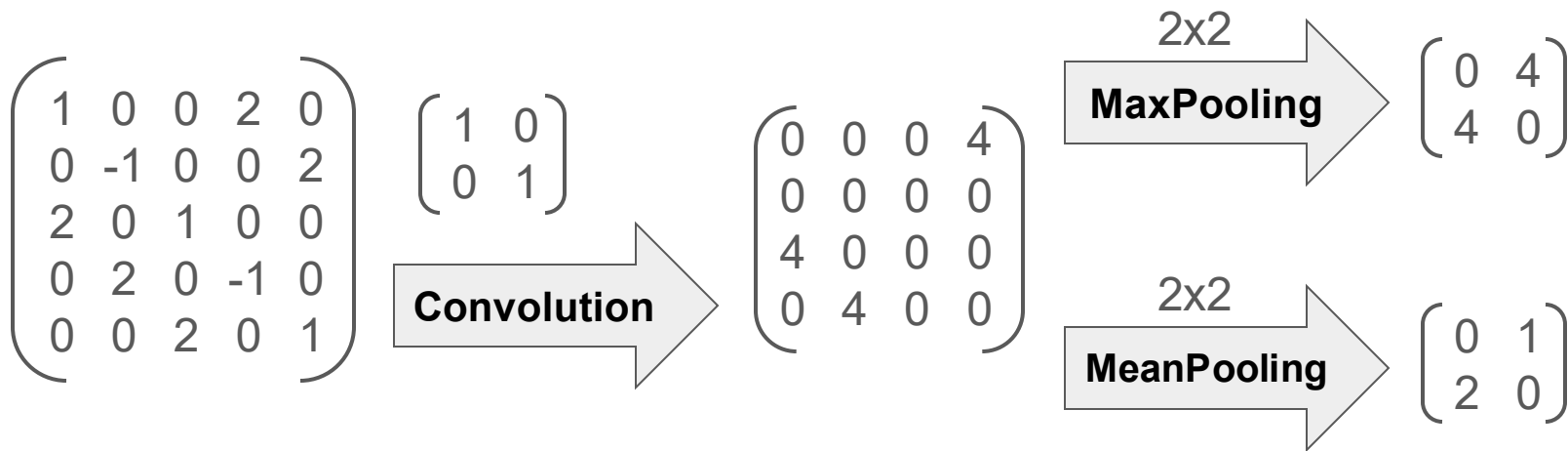
$$\text{data} = \begin{pmatrix} 1 & 0 & 0 & 2 & 0 \\ 0 & -1 & 0 & 0 & 2 \\ 2 & 0 & 1 & 0 & 0 \\ 0 & 2 & 0 & -1 & 0 \\ 0 & 0 & 2 & 0 & 1 \end{pmatrix} \quad \text{fa} = \begin{pmatrix} 1 & 0 \\ 0 & 1 \end{pmatrix} \quad \text{fb} = \begin{pmatrix} 1 & 0 \\ 0 & -1 \end{pmatrix}$$

- Compute the convolution and 2x2 max-pooling outputs
- Compute the convolution and 2x2 mean-pooling outputs



Convolution and pooling exercise...

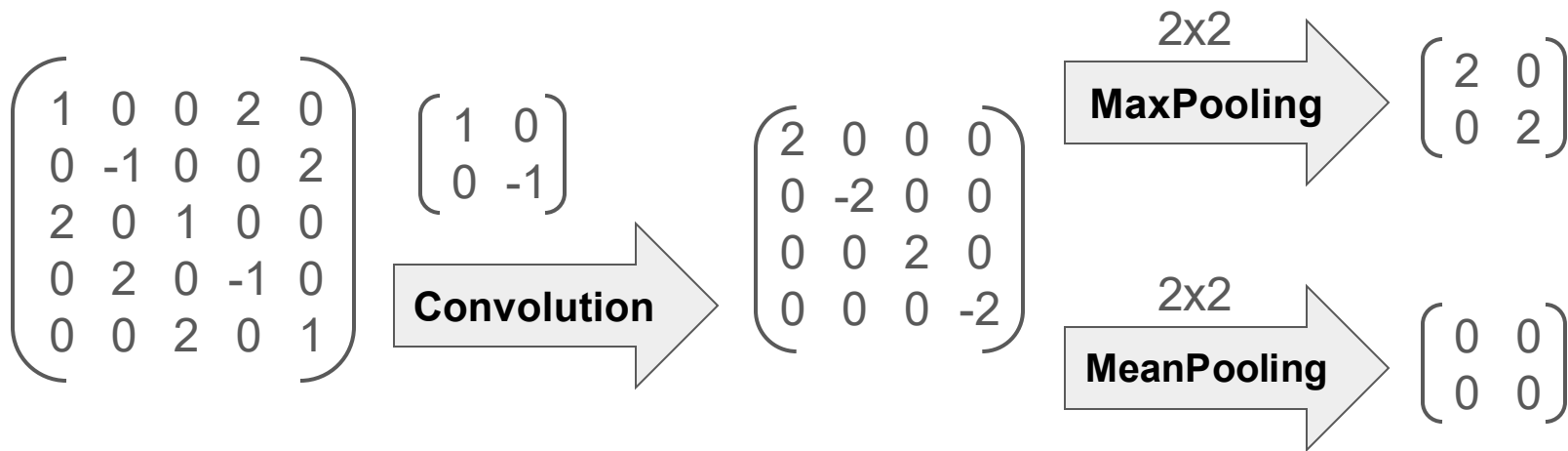
- Convolution with filter a and 2x2 max- and mean-pooling





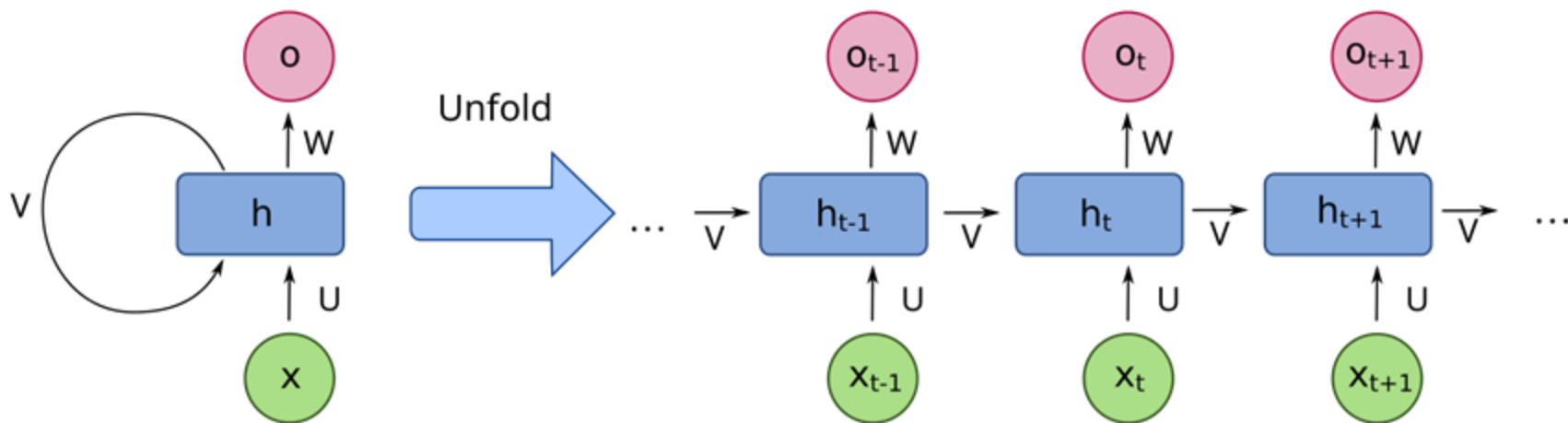
Convolution and pooling exercise...

- Convolution with filter b and 2x2 max- and mean-pooling



Accounting for temporal dependencies

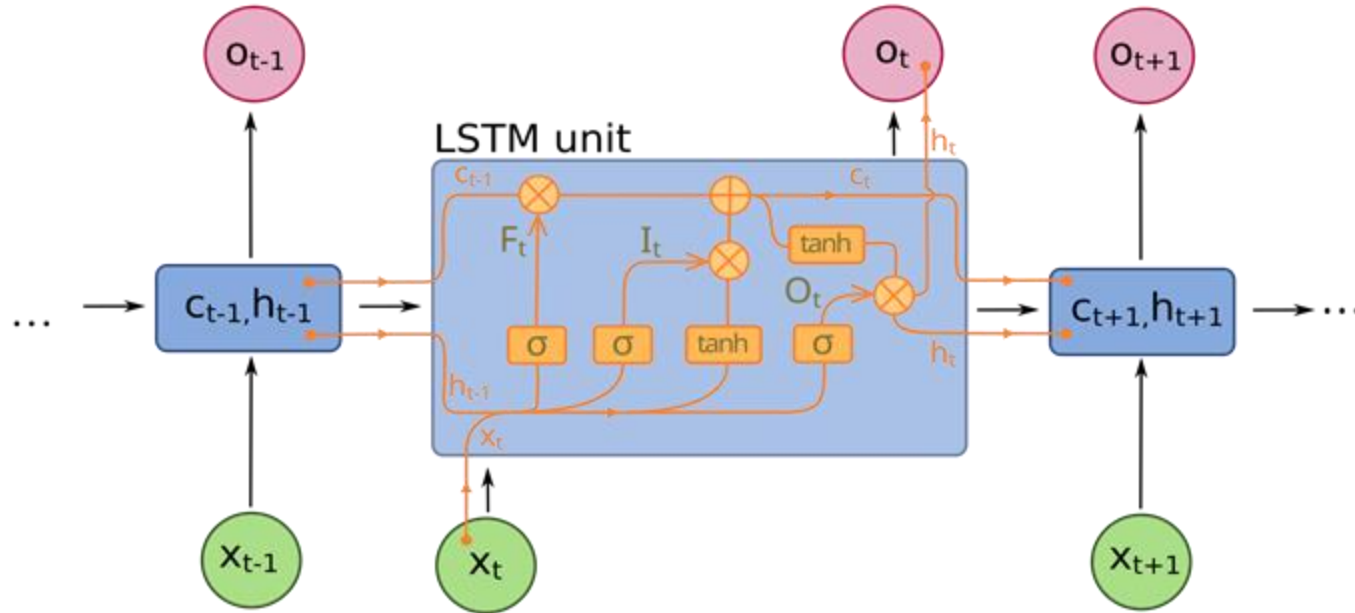
- Recurrent Neural Networks (RNNs):
 - Hidden layer with feedback loop: $\mathbf{o}_t = \mathbf{w}^T \mathbf{h}(\mathbf{u}^T \mathbf{x}_t + \mathbf{v}^T \mathbf{h}_{t-1})$



https://en.wikipedia.org/wiki/Recurrent_neural_network

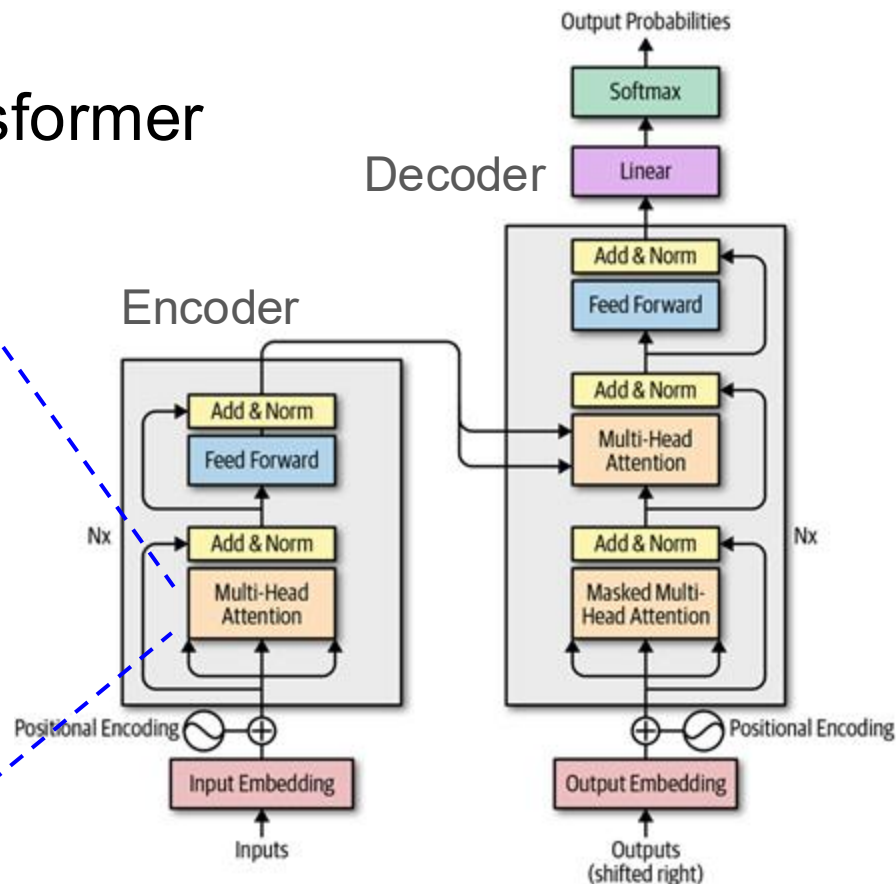
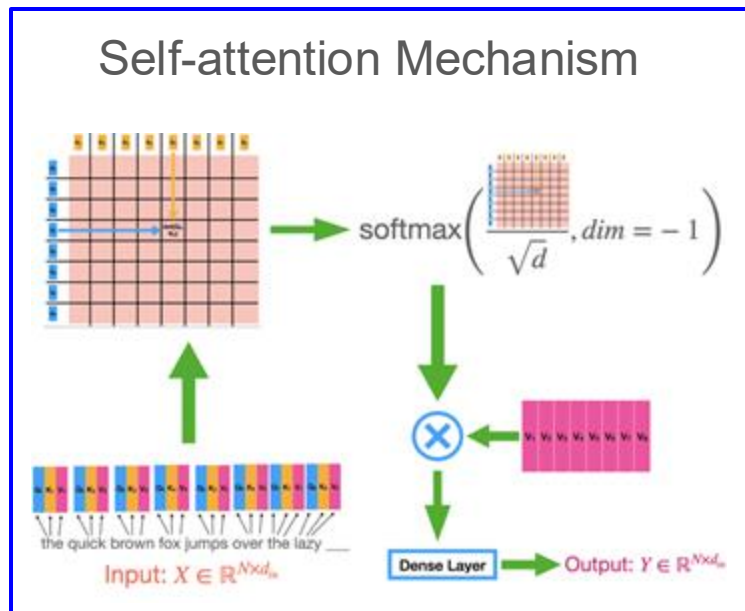
Addressing the vanishing gradient problem in RNNs

- Long short-term memories (LSTMs):



https://en.wikipedia.org/wiki/Recurrent_neural_network

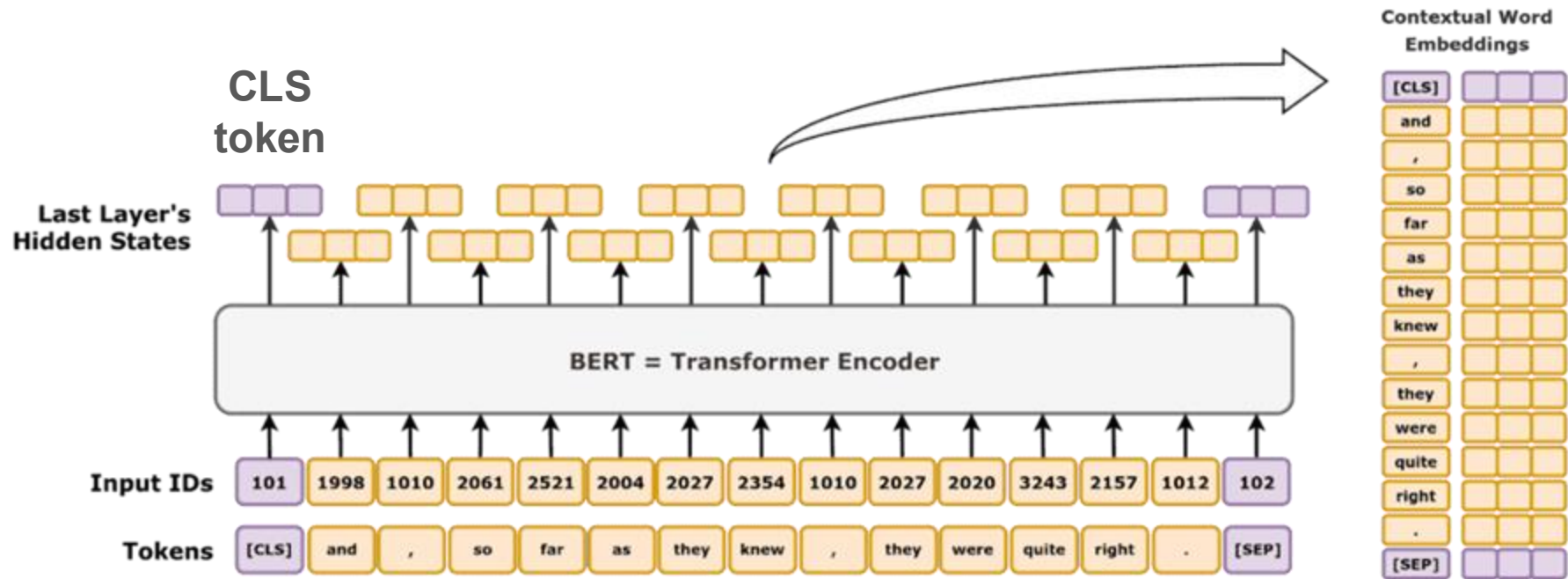
Beyond recurrence: the transformer



<https://learnopencv.com/attention-mechanism-in-transformer-neural-networks/>

https://www.researchgate.net/figure/Transformer-architecture-with-its-encoder-left-and-decoder-right-Vaswani-et-al_fig1_366569196

BERT: bidirectional encoder representations from transformers



<https://www.geeksforgeeks.org/nlp/explanation-of-bert-model-nlp/>



Hands-on Coding Session W9S1

Hands on coding session W9S1



- Get familiar with the notebook for W9S1 and its contents
- Install the transformers and pytorch modules
- Look at the documentation for functions used there:
 - Scikit-Learn: <https://scikit-learn.org/0.16/index.html#>
 - Transformers: <https://huggingface.co/docs/transformers/en/index>
- Work on exercises in Part 1: Classification with MLPs
- Work on exercises in Part 2: Regression with MLPs
- Work on exercises in Part 3: BERT and text classification

Week 9 - Closing remarks

Wrap-up and Homework 9

- What did we look at today?
- Further reading and support materials
- Ninth homework assignment and project presentations

What did we look at today?

Neural Networks	● Multilayer Perceptron ● Backpropagation ● Gradient Descent Algorithms
Deep Learning	● Convolutional Neural Networks ● Recurrent Neural Networks ● The Transformer
Hands-on coding session W9S1	● Classification with MLPs ● Regression with MLPs ● BERT and text classification

Further reading and support materials

- James, Witten, Hastie, Tibshirani, Taylor, “An Introduction to Statistical Learning with Applications in Python” (Chapter 10)
<https://www.statlearning.com/>
- Abu-Mostafa, Magdon-Ismael, Lin, “Learning From Data”
(Chapter 7) <https://work.caltech.edu/telecourse>

Ninth homework assignment (due date 08/31)

- Complete all coding exercises from coding session W9S1



Project presentations on Thursday, August 28th

- 20 mins per team: 15 mins presentation + 5 mins Q&A
- Your presentation must include:
 - Introduction and motivation: what is your project about? any previous related works? what is the impact/benefit of it?
 - Results and analysis: what datasets, models, and metrics did you used? what results did you obtain and what they mean?
 - Cost analysis: how much it costs to run your solution at scale?
 - Conclusions and future work: what are the main conclusions of your project? what would you do next if you can continue it?

Project slides and report on Thursday, August 28th

- Submit your team's presentation deck (pdf format!) into Canvas
- Submit your team's no-more-than-five-pages report (pdf format!) into Canvas. The report must include the same sections in the deck:
 - Introduction and motivation
 - Results and analysis
 - Cost analysis
 - Conclusions and future work
- You can use LLM-based technologies to write your essay, but need to attach all prompt interaction in an appendix



Remember:

Office Hours

Tuesday 08/26 6:00-7:00pm

Next Class

Thursday 08/28, 5:30-9:30pm

Ninth Homework Due

Sunday 08/31, 11:59pm